

**BEYOND CONVERGENCE: A LIGHTWEIGHT
HUMAN-IN-THE-LOOP SEMANTIC CONFLICT
TRIAGE FRAMEWORK FOR OFFLINE-FIRST CRDT-
BASED COLLABORATIVE TEXT EDITING**

A PROJECT REPORT

Submitted by

RAM PRATHEESH S K	230701258
NAVEEN V	230701207
TARUN P	230701401
ATHIRA D R	230701047

*in partial fulfillment for the award of the degree
of*

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING**



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY : CHENNAI 600 025

SEPTEMBER 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “BEYOND CONVERGENCE: A LIGHTWEIGHT HUMAN-IN-THE-LOOP SEMANTIC CONFLICT TRIAGE FRAMEWORK FOR OFFLINE-FIRST CRDT-BASED COLLABORATIVE TEXT EDITING” is the bonafide work of “RAM PRATHEESH S K (230701258), NAVEEN V (230701207), TARUN P (230701401) and ATHIRA D R (230701047)” who carried out the project work under my supervision.

SIGNATURE

Dr. J. Manoranjini

HEAD OF THE DEPARTMENT

Department of Computer Science and
Engineering
Rajalakshmi Engineering College
Chennai - 602 105

SIGNATURE

Mr. J. Balachandar

SUPERVISOR

Assistant Professor
Department of Computer Science and
Engineering
Rajalakshmi Engineering College
Chennai - 602 105

Submitted for the Project Viva-Voce Examination held on
_____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Offline-first collaboration must preserve work during network loss while still producing a semantically usable shared document. This project implements CollabX, a real-time collaborative rich-text editor using React, TypeScript, TipTap/ProseMirror, Yjs CRDTs, IndexedDB, Node.js/Express, WebSockets and MongoDB. Local edits remain available offline and synchronize automatically after reconnection.

The final system also introduces a location-based Inspection workspace. Collaborators record observations that retain author and connectivity metadata. Observation pairs within a location are classified as duplicate, complementary, contradictory or independent by a schema-validated Gemini service. The AI does not determine factual truth: contradictions require human review, while complementary findings may receive a combined-text suggestion. Review decisions are persisted for auditability.

PDF export remains locked until the owner and all collaborators finalize the inspection. The exported report groups finalized observations by location and preserves reviewed conflict outcomes. Thus, the system separates CRDT synchronization correctness from human-governed semantic reconciliation while retaining offline-first availability.

ACKNOWLEDGEMENT

We express our sincere gratitude to the management and the Principal of Rajalakshmi Engineering College for providing the facilities and academic environment required to carry out this project. We are deeply thankful to Dr. J. Manoranjini, Head of the Department, Computer Science and Engineering, for the encouragement and departmental support extended throughout the course of the work.

We place on record our sincere gratitude to our project supervisor, Mr. J. Balachandar, Assistant Professor, Department of Computer Science and Engineering, for his guidance, constructive feedback and continued support during the design, implementation, testing and documentation of the project. We also thank the project coordination team and faculty reviewers for the suggestions provided during periodic reviews.

Finally, we thank our families, friends and classmates for their support and encouragement throughout the project.

RAM PRATHEESH S K

NAVEEN V

TARUN P

ATHIRA D R

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1.	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 EXISTING SYSTEM	4
	1.5 PROPOSED SYSTEM	4
2.	LITERATURE SURVEY	6
	2.1 LITERATURE REVIEW	6
	2.1.1 Operational Transformation	6
	2.1.2 Commutative and Conflict-free Replication	6
	2.1.3 Local-first Software	7
	2.1.4 Rich-text CRDTs	7
	2.1.5 Semantic Conflict Models	8
	2.1.6 Revision Intent Analysis	8
	2.1.7 Natural Language Inference	8
	2.1.8 Human-AI Revision	9
	2.2 COMPARISON AND DISCUSSION	10
	2.3 CONCLUSION	11
3.	SYSTEM DESIGN	12
	3.1 SYSTEM ARCHITECTURE	12
	3.2 SYSTEM REQUIREMENTS	13
	3.2.1 Software Requirements	13
	3.2.2 Hardware Requirements	14
	3.3 DATA AND PERSISTENCE DESIGN	14

	3.4 SYSTEM WORKFLOW	15
	3.4.1 Offline Editing and Reconnection	15
	3.4.2 Inspection Collection and Review	15
	3.4.3 Finalization and PDF Export	16
	3.5 SEMANTIC ANALYSIS DESIGN	18
	3.6 API ARCHITECTURE	19
4.	PROJECT DESCRIPTION	20
	4.1 METHODOLOGY	20
	4.2 SYSTEM MODULES	21
	4.3 DETAILED INSPECTION FEATURE	22
	4.3.1 Inspection Workspace and Location Model	22
	4.3.2 Observation Capture and Connectivity Metadata	23
	4.3.3 Complementary Observation Handling	23
	4.3.4 Contradiction Handling and Human Decision	24
	4.3.5 Collaborator Finalization and Export Lock	25
	4.3.6 Exported Inspection Report	26
5.	IMPLEMENTATION AND RESULT DISCUSSION	28
	5.1 IMPLEMENTATION	28
	5.2 EXPERIMENTAL SETUP	30
	5.3 COLLABORATION TEST SCENARIOS	31
	5.4 SYNCHRONIZATION RESULTS	31
	5.5 INSPECTION FEATURE FUNCTIONAL VALIDATION	32
	5.6 SEMANTIC RESEARCH EVALUATION FROM THE APPROVED PAPER	33
	5.7 DISCUSSION	34
	5.8 LIMITATIONS	35
6.	CONCLUSION AND FUTURE WORK	36
	6.1 CONCLUSION	36
	6.2 FUTURE WORK	37
	APPENDIX 1 - INSPECTION DATA MODEL AND STATUS RULES	38
	APPENDIX 2 - DETAILED TEST CHECKLIST	39
	REFERENCES	40

LIST OF TABLES

Table 2.1 Comparison of Related Approaches	10
Table 3.1 Software Requirements.....	13
Table 3.2 Hardware Requirements	14
Table 3.3 Core Persistence Entities	14
Table 3.4 Semantic Relationship Taxonomy and Action Policy	18
Table 3.5 Major Inspection API Operations	19
Table 5.1 Test Environment Configuration	30
Table 5.2 Collaboration and Offline Test Scenarios	31
Table 5.3 Scalability Test Results from the Existing Draft	32
Table 5.4 Inspection Workflow Functional Validation.....	32
Table 5.5 Approved-Paper Semantic Triage Evaluation.....	34
Table A.1 Inspection Rules Implemented in Source Code	38
Table B.1 End-to-End Project Verification Checklist.....	39

LIST OF FIGURES

Figure 3.1 Overall System Architecture	13
Figure 3.2 Inspection Collection and Semantic Review Workflow	16
Figure 3.3 Inspection Finalization and PDF Export Gate	17
Figure 3.4 Research Semantic-Triage Cascade from the Approved Paper	18
Figure 4.1 Inspection Workspace with Location-Based Observations	24
Figure 4.2 Complementary Semantic Analysis and Suggested Combined Text	24
Figure 4.3 Potential Contradiction Requiring Human Review	25
Figure 4.4 Contradictory Observations Retained by Human Decision	25
Figure 4.5 Complete Multi-User Finalization with PDF Export Enabled	26
Figure 4.6 Sample Exported Inspection PDF	27
Figure 5.1 Dashboard Interface.....	28
Figure 5.2 Collaborative Rich-Text Editor Interface	29
Figure 5.3 AI-Generated Change Summary Panel	29
Figure 5.4 Diagnostics Panel	30

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
API	Application Programming Interface
CRDT	Conflict-free Replicated Data Type
DB	Database
HITL	Human-in-the-Loop
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
LLM	Large Language Model
MD5	Message Digest Algorithm 5
NLI	Natural Language Inference
NLP	Natural Language Processing
OAuth	Open Authorization
OT	Operational Transformation
REST	Representational State Transfer
SBERT	Sentence-BERT
UI	User Interface
Yjs	JavaScript CRDT framework

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Real-time collaborative editing has become a normal expectation in document-oriented applications. Multiple users expect to work in a shared space, observe collaborator presence, view edits quickly and return to a document without losing work. Conventional collaborative architectures often assume that the synchronization service remains reachable. When a network path becomes unstable, the user experience becomes fragile because a server-authoritative application may delay edits, fall back to an isolated copy or require explicit conflict resolution when the connection returns.

Offline-first and local-first design reverse this assumption. A user-visible local replica is treated as the working state rather than as a temporary cache. Edits are accepted locally, persisted on the device and synchronized when communication becomes available. Conflict-free Replicated Data Types provide a mathematical basis for this approach because concurrent operations can be merged while guaranteeing convergence of replicas. Yjs provides a practical CRDT implementation for collaborative text and integrates with ProseMirror/TipTap for rich-text editing [3]-[9].

This project implements these ideas in a browser-based collaborative editor and then extends them with a separate semantic review problem. CRDT convergence answers the question “do all replicas end with the same state?” It does not answer whether two sentences written concurrently are compatible in meaning. A converged document may still contain duplicate claims, complementary details that should be combined, or direct contradictions. The approved research paper therefore treats semantic uncertainty as a post-convergence concern and preserves human control over disputed meaning [10]-[20].

The final project adds a practical Inspection workspace that makes this distinction visible in a concrete application. An inspection is organized by locations. Collaborators add observations to each location, and the system compares observations within that location. The relationship is classified as duplicate, complementary, contradictory or independent. Explanations and suggestions are shown to reviewers, but the semantic service is prohibited from determining which inspector is factually correct. A human decision is recorded before the workflow can proceed to finalization.

The Inspection module also introduces a multi-user approval gate. Individual collaborators can finalize only after required semantic reviews are complete. The report remains only partially finalized until every required participant has finalized. PDF export is then unlocked and the exported report records observations and review outcomes. This workflow demonstrates how collaborative synchronization, AI-assisted semantic analysis and accountable human review can be combined without allowing the language model to override the underlying CRDT state.

1.2 PROBLEM STATEMENT

Existing collaborative editors primarily focus on synchronizing state and providing low-latency multi-user interaction. Under intermittent connectivity, conventional server-authoritative designs can prevent editing or require users to reconcile divergent copies manually. CRDT-based local-first editors improve availability and merge correctness, but their guarantee is structural rather than semantic: every replica may converge to the same text while that text still contains incompatible statements [5], [6].

In inspection-oriented work this limitation becomes important. Two inspectors may describe the same location while disconnected. After synchronization, both statements can be preserved correctly by the CRDT even when one says that an element is damaged and the other says that it is normal.

Blindly accepting both statements into a final report is undesirable, but automatically allowing an AI model to delete or rewrite disputed human observations is also unsafe. Therefore the project requires a mechanism that preserves offline work, detects semantic relationships after synchronization, exposes uncertainty, records human decisions and prevents formal report export until review and finalization are complete.

1.3 OBJECTIVES

- Design and implement an offline-first, real-time collaborative rich-text editor using Yjs CRDTs and TipTap/ProseMirror.
- Persist local document state in browser IndexedDB so editing can continue during network loss and survive refresh or reopening.
- Provide automatic WebSocket reconnection and state-vector-based resynchronization while preserving collaborator awareness and shared document isolation.
- Implement authentication, document management, sharing, server-side persistence, diagnostics and AI-assisted change summaries.
- Introduce a location-based Inspection workspace in which multiple collaborators can create observations and work across online/offline connectivity states.
- Classify related inspection observations using the four semantic relationships: duplicate, complementary, contradictory and independent.
- Keep semantic judgments auditable by storing relationship, confidence, reason, suggested action, content hashes, human decision and decision metadata.
- Enforce a human-in-the-loop finalization workflow in which unresolved semantic reviews block finalization and PDF export remains locked until all required users finalize.

- Evaluate CRDT synchronization using the existing test scenarios and relate the implemented Inspection workflow to the semantic-triage research evaluation reported in the approved paper.

1.4 EXISTING SYSTEM

Traditional collaborative systems use centralized sequencing, locking or Operational Transformation to keep users synchronized. Operational Transformation has been highly influential, but transformation functions become increasingly complex as operation types and rich-text structures grow [1]. Server-centric systems also depend on continuous access to the authoritative state and may offer limited behavior when clients are fully disconnected.

CRDT-based editors improve these limitations by allowing replicas to accept local operations and merge them deterministically [2]-[4]. Modern local-first work shows that this model improves availability and data ownership [5], while rich-text CRDTs such as Peritext, Eg-walker and Fugue address formatting intent and interleaving behavior [7]-[9]. These mechanisms still do not determine whether two natural-language statements are editorially compatible.

Existing NLP approaches provide semantic similarity, natural-language inference, revision-intent analysis and factual verification [12]-[16]. Human-AI writing systems further show why final editorial control should remain visible to the user [17]-[20]. The gap addressed by the project is the integration of these ideas with a working offline-first collaborative application and an explicit review/finalization workflow.

1.5 PROPOSED SYSTEM

The proposed system combines a CRDT-based collaborative editor with an application-level semantic review layer. The core editor remains responsible for synchronization and never delegates merge semantics to an AI model. The

semantic layer observes structured inspection observations after they have been captured and compares candidate pairs only as an advisory layer. This separation ensures that model uncertainty cannot corrupt the replication protocol.

For Inspection, the browser presents a location tree, observation cards, collaborator state, semantic reconciliation results and report status. The backend stores inspection metadata, observation snapshots, semantic results and research logs in MongoDB. A Gemini service classifies observation pairs and returns structured JSON. The response is sanitized before persistence. Contradictions trigger human review, complementary pairs can provide combined wording, duplicate pairs can suggest keeping one, and independent observations are preserved.

The finalization gate provides an additional integrity control. A user cannot finalize while a non-independent semantic result remains pending. Each editor-level participant finalizes individually, and the inspection status changes to `FINALIZED` only after the owner and all unique collaborators have finalized. PDF export becomes `AVAILABLE` only at that stage. This creates a clear progression from collection to semantic review, human decision, individual approval and final report generation.

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE REVIEW

2.1.1 Operational Transformation

Sun and Ellis analyzed consistency issues in real-time group editors and established important foundations for Operational Transformation [1]. OT modifies operations according to concurrently executed operations so that replicas can preserve user intent. The approach is effective but transformation logic is coupled to operation types and can become difficult to maintain in complex rich-text structures.

The relevance of this work to the present project is that it contributes one part of the overall pipeline—replication, semantic representation, inference, verification or human interaction. The project does not claim novelty for any individual component; its contribution lies in integrating these concerns into an offline-first collaborative editor and exposing the semantic decision process to users.

2.1.2 Commutative and Conflict-free Replication

Preguiça et al. introduced commutative replicated approaches for cooperative editing [2], and Shapiro et al. formalized CRDTs as replicated data types whose operations or states can be combined without centralized conflict resolution while converging to the same value [3]. Kleppmann and Beresford later demonstrated conflict-free replicated JSON structures suitable for structured application data [4].

The relevance of this work to the present project is that it contributes one part of the overall pipeline—replication, semantic representation, inference, verification or human interaction. The project does not claim novelty for any individual component; its contribution lies in integrating these concerns into an

offline-first collaborative editor and exposing the semantic decision process to users.

2.1.3 Local-first Software

Kleppmann et al. described local-first software as an application model in which a local copy remains primary and cloud synchronization supports collaboration rather than making the network mandatory [5]. Schiefer et al. later highlighted that some application invariants cannot simply be merged, establishing the important distinction between technical mergeability and application-level validity [6].

The relevance of this work to the present project is that it contributes one part of the overall pipeline—replication, semantic representation, inference, verification or human interaction. The project does not claim novelty for any individual component; its contribution lies in integrating these concerns into an offline-first collaborative editor and exposing the semantic decision process to users.

2.1.4 Rich-text CRDTs

Peritext encodes rich-text formatting spans and editing operations using CRDT techniques that better preserve formatting intent [7]. Eg-walker studies efficient collaborative text editing across long histories and offline branches [8]. Fugue addresses undesirable interleaving of concurrent edits while maintaining convergence [9]. These works improve structural behavior but do not alone determine whether two natural-language claims are mutually compatible.

The relevance of this work to the present project is that it contributes one part of the overall pipeline—replication, semantic representation, inference, verification or human interaction. The project does not claim novelty for any individual component; its contribution lies in integrating these concerns into an offline-first collaborative editor and exposing the semantic decision process to users.

2.1.5 Semantic Conflict Models

Semenov and Aksenov model semantic dependencies in collaborative data structures [10]. Their work demonstrates that semantic conflict is not a new idea in replicated systems; however, the project focuses on content-level relations between natural-language observations after synchronization rather than changing CRDT merge rules. Gu et al. earlier applied ontology-based techniques to semantic conflicts in collaborative design documents [11].

The relevance of this work to the present project is that it contributes one part of the overall pipeline—replication, semantic representation, inference, verification or human interaction. The project does not claim novelty for any individual component; its contribution lies in integrating these concerns into an offline-first collaborative editor and exposing the semantic decision process to users.

2.1.6 Revision Intent Analysis

Yang et al. classified semantic edit intentions in Wikipedia revisions, including fact updates, copy editing, elaboration and verifiability [12]. These intent labels explain why an edit was made but do not directly describe the relationship between two concurrent edits. The approved paper uses intent as a risk prior and sampling aid rather than as the four-class conflict label.

The relevance of this work to the present project is that it contributes one part of the overall pipeline—replication, semantic representation, inference, verification or human interaction. The project does not claim novelty for any individual component; its contribution lies in integrating these concerns into an offline-first collaborative editor and exposing the semantic decision process to users.

2.1.7 Natural Language Inference

The SNLI corpus provides entailment, contradiction and neutral labels for sentence pairs [13], while MultiNLI broadens inference evaluation to multiple

genres [14]. Sentence-BERT provides efficient sentence embeddings and cosine similarity suitable for retrieving semantically related pairs [15]. FEVER provides a factual verification setting in which claims may be supported or refuted by evidence [16]. Together these resources motivate staged semantic comparison.

The relevance of this work to the present project is that it contributes one part of the overall pipeline—replication, semantic representation, inference, verification or human interaction. The project does not claim novelty for any individual component; its contribution lies in integrating these concerns into an offline-first collaborative editor and exposing the semantic decision process to users.

2.1.8 Human-AI Revision

CoAuthor records human and language-model contributions during collaborative writing [17], and Re3 models document revision more broadly [18]. RARR and SelfCheckGPT study revision, verification and self-checking of language-model output [19], [20]. Their limitations reinforce a central design principle of the present project: AI-generated reasoning and suggestions must remain reviewable rather than silently replacing disputed human content.

The relevance of this work to the present project is that it contributes one part of the overall pipeline—replication, semantic representation, inference, verification or human interaction. The project does not claim novelty for any individual component; its contribution lies in integrating these concerns into an offline-first collaborative editor and exposing the semantic decision process to users.

2.2 COMPARISON AND DISCUSSION

Table 2.1 Comparison of Related Approaches

APPROACH	PRIMARY STRENGTH	LIMITATION RELATIVE TO THIS PROJECT
-----------------	-------------------------	--

APPROACH	PRIMARY STRENGTH	LIMITATION RELATIVE TO THIS PROJECT
Operational Transformation [1]	Real-time concurrent operation transformation	Complex transformations; offline/local-first behavior is not the main focus
CRDTs [2]-[4]	Deterministic convergence without central conflict resolution	Convergence does not guarantee semantic correctness
Local-first software [5], [6]	Availability and locally writable replicas	Application invariants may still require explicit reconciliation
Rich-text CRDTs [7]-[9]	Preservation of rich-text structure and reduced interleaving	No natural-language contradiction judgment
Semantic conflict/revision models [10]-[12]	Application semantics and revision intention	Not a complete offline-first inspection workflow
SBERT/NLI/verification [13]-[16]	Efficient semantic relation features	Model output still needs application policy and review
Human-AI revision [17]-[20]	Reviewer-centered AI assistance	Not directly tied to CRDT reconnection and finalization

The literature reveals two distinct notions of correctness. Replication correctness concerns causality, update preservation and convergence. Editorial correctness concerns whether the resulting human-authored statements are redundant, compatible or contradictory. The project deliberately does not merge these responsibilities. Yjs remains the authority for replicated state, while the semantic layer is advisory and operates on stored observation snapshots.

The approved paper describes a lightweight cascade that uses structural pruning, lexical features, Sentence-BERT, bidirectional NLI, selective LLM escalation and human verification. The final application preserves the paper’s four-class taxonomy and human-governance policy, but the implemented Inspection service currently performs lightweight pair filtering followed by a structured Gemini classification. This distinction is important: the report documents the implemented final system without representing research-stage cascade components as already deployed.

2.3 CONCLUSION

Prior research provides mature solutions for replicated state and a rich set of NLP techniques for semantic comparison, yet the integration of offline-first rich-text collaboration, domain-specific inspection observations, explicit semantic relation labels, human review and multi-user finalization remains underexplored. The proposed implementation addresses this integration gap while keeping synchronization correctness and semantic judgment clearly separated.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The final system uses a browser client, an Express/TypeScript backend, a native WebSocket synchronization layer, MongoDB/Mongoose persistence and the Gemini API. The React client contains the dashboard, rich-text editor and Inspection workspace. TipTap/ProseMirror is bound to a Yjs document, and y-indexeddb persists local CRDT state in the browser. Zustand and React Query manage application state and server data. The WebSocket layer exchanges Yjs updates, state vectors and awareness data so that replicas can synchronize efficiently when connected.

The backend separates document synchronization from application APIs. REST endpoints provide authentication, document management, Inspection CRUD, location management, observation snapshots, semantic-analysis triggers, human decisions, report status and export data. MongoDB stores users, document state, inspections, observation snapshots, semantic results, change logs and research logs. The semantic service calls Gemini only for application-level analysis; it does not modify Yjs operations or their merge order.

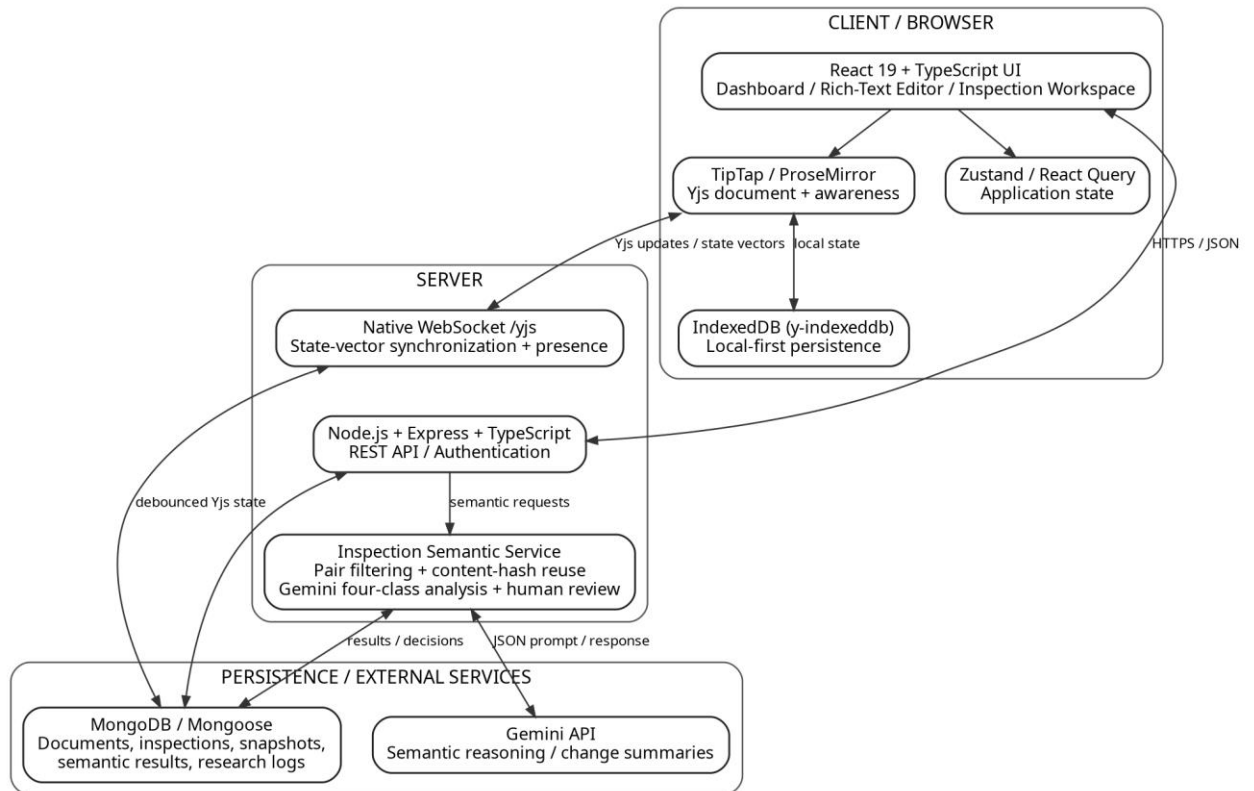


Figure 3.1 Overall System Architecture

3.2 SYSTEM REQUIREMENTS

3.2.1 Software Requirements

Table 3.1 Software Requirements

COMPONENT	TECHNOLOGY / REQUIREMENT
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4
Rich-text editor	TipTap v3 / ProseMirror
Client state	Zustand and React Query
CRDT	Yjs with y-indexeddb and awareness protocols
Real-time transport	Native WebSocket
Backend	Node.js, Express, TypeScript
Database	MongoDB with Mongoose
Authentication	JWT, bcrypt and Google OAuth
AI integration	Gemini API for change summaries and Inspection semantic analysis
Browser	Modern Chromium/Edge-class browser with IndexedDB and WebSocket support

3.2.2 Hardware Requirements

Table 3.2 Hardware Requirements

RESOURCE	RECOMMENDED CONFIGURATION
Developer/Test machine	Modern multi-core processor
Memory	8 GB minimum; 16 GB or higher recommended
Storage	Sufficient local storage for dependencies, browser cache and test data
Network	LAN/Wi-Fi/Internet for synchronization and cloud services; application supports temporary disconnection
Client device	Desktop/laptop browser; responsive access from compatible mobile browser

3.3 DATA AND PERSISTENCE DESIGN

The collaborative editor uses two complementary persistence layers. Yjs document state is stored locally in browser IndexedDB, allowing a client to reopen the document and retain unsynchronized work. The server also stores document state in MongoDB so that newly connected clients can bootstrap the shared document. Synchronization uses Yjs state vectors to identify missing updates instead of retransmitting the entire document history.

The Inspection feature uses explicit MongoDB models because its workflow requires queryable business state in addition to shared editing state. The Inspection model stores title, description, owner, collaborators, status, a Yjs-state buffer, a backup list of locations, deletion state and finalization metadata. The Observation model stores inspection ID, observation ID, location ID, author identity, plain-text content, online/offline connectivity state and timestamps. SemanticResult stores the compared observation IDs, content hashes, relationship, confidence, reason, suggested action/text, human decision, reviewer identity, decision time and AI processing time.

Table 3.3 Core Persistence Entities

ENTITY	IMPORTANT FIELDS	PURPOSE
Inspection	owner, collaborators, status, locations,	Controls inspection lifecycle and

ENTITY	IMPORTANT FIELDS	PURPOSE
	finalizedBy, finalizedAt	multi-user finalization
Observation	locationId, authorId, authorName, plainTextContent, connectivityState	Stores a queryable snapshot of each field observation
SemanticResult	observationIds, contentHashes, relationship, confidence, humanDecision	Stores AI relation output and human resolution
ResearchLog	eventType, data, timestamp	Maintains an auditable trace of inspection and review events
Document / Yjs state	document state and synchronization metadata	Supports collaborative rich-text convergence and server persistence

3.4 SYSTEM WORKFLOW

3.4.1 Offline Editing and Reconnection

A collaborator opens a document and works against a locally writable Yjs replica. Local updates immediately affect the visible editor and are persisted through y-indexeddb. If the WebSocket is unavailable, editing continues without waiting for a round trip. On reconnection, the client and server exchange synchronization messages and state-vector information; missing CRDT updates are exchanged and Yjs merges them deterministically. Awareness information is refreshed separately so presence and cursor state do not become part of persistent document content.

3.4.2 Inspection Collection and Review

The Inspection workspace begins with creation of an inspection and one or more named locations. Collaborators add observations under those locations. Observation snapshots are upserted to MongoDB and include the recorded connectivity state. Semantic analysis is triggered per location after at least two observations are available. Candidate pairs are generated from substantive observations, and the relationship result is persisted for review. Decisions are broadcast to the inspection room so other collaborators see updated review state.

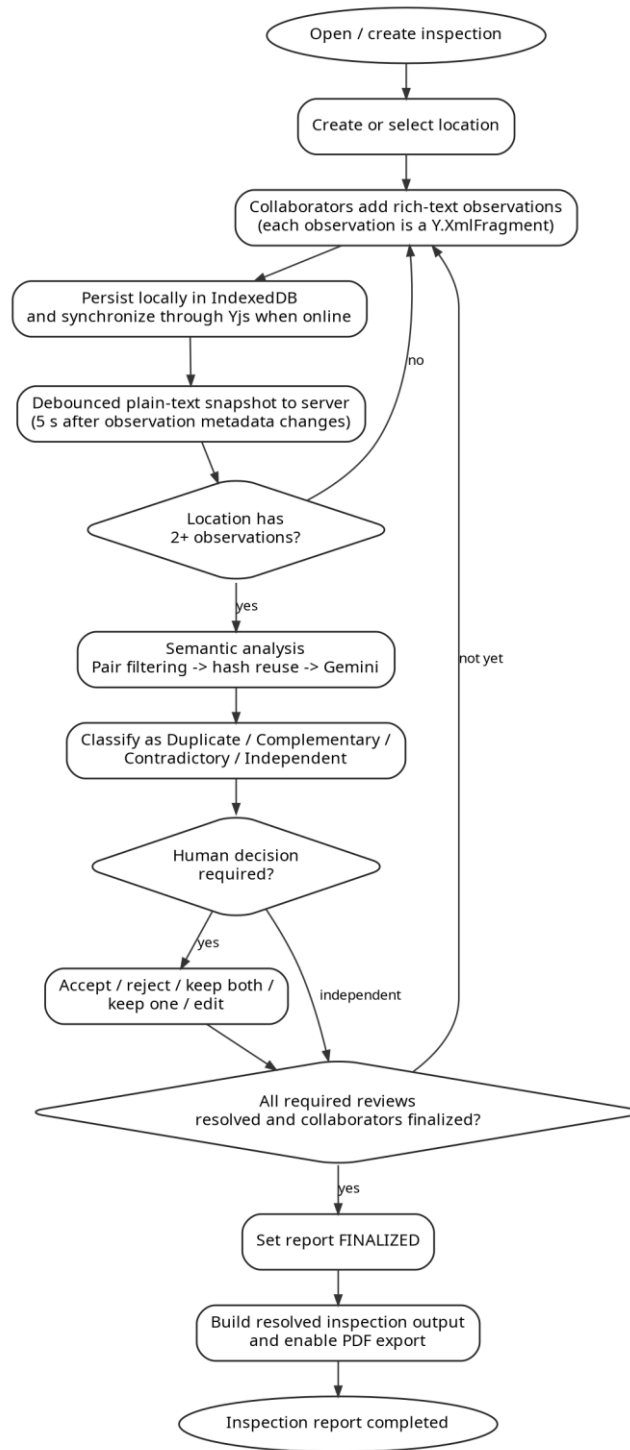


Figure 3.2 Inspection Collection and Semantic Review Workflow

3.4.3 Finalization and PDF Export

Finalization is intentionally separated from semantic analysis. The backend counts non-independent SemanticResult records that remain PENDING. If any exist, finalization is rejected. When reviews are complete, each editor-level participant can finalize. The backend records the user ID in finalizedBy,

deduplicates collaborator identities and compares the finalized count with the owner plus unique collaborators. A partially approved inspection remains `READY_TO_FINALIZE`, while complete approval changes the inspection status to `FINALIZED` and records `finalizedAt`. Report status exposes PDF Export as `AVAILABLE` only for the effectively finalized state.

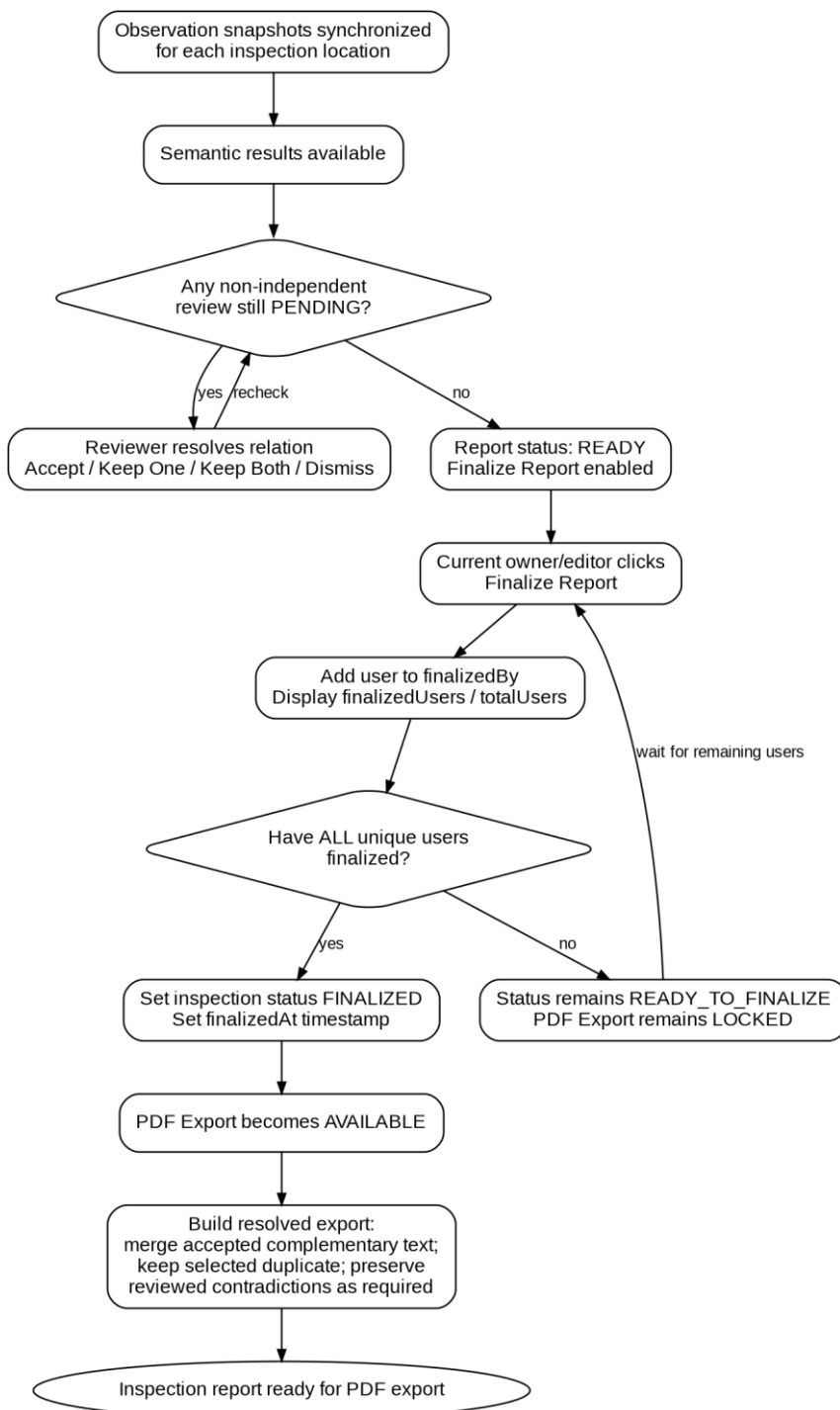


Figure 3.3 Inspection Finalization and PDF Export Gate

3.5 SEMANTIC ANALYSIS DESIGN

The implemented semantic service compares two observations from the same inspection location. The request sent to Gemini contains the location name, both authors and both observation texts. The prompt defines exactly four valid relationships: DUPLICATE, COMPLEMENTARY, CONTRADICTORY and INDEPENDENT. It explicitly instructs the model not to decide factual truth. Complementary results may return combined text, contradictions must return HUMAN_REVIEW, duplicates suggest KEEP_ONE and independent observations suggest KEEP_BOTH.

The raw model response is parsed as JSON and validated before use. Unknown relationship values fall back to INDEPENDENT, unknown actions fall back to KEEP_BOTH, confidence is clamped to the range 0 to 1, explanations are length-limited and suggested text is sanitized. The service retries HTTP 503 failures up to five times with exponential backoff. These controls reduce the chance that malformed or transient model output breaks the application workflow.

Before model invocation, observation text shorter than five characters is filtered out. Pair generation then produces combinations of substantive observations. An MD5 hash of normalized observation content is stored with each semantic result; this supports reuse/invalidation when content changes and prevents stale results from silently representing edited observations. When observations are deleted, related semantic results are also removed.



Figure 3.4 Research Semantic-Triage Cascade from the Approved Paper

Table 3.4 Semantic Relationship Taxonomy and Action Policy

RELATION	MEANING	DEFAULT SYSTEM ACTION
DUPLICATE	Same or nearly same finding without significant additional information	KEEP_ONE suggestion; human can review
COMPLEMENTARY	Compatible findings that jointly	COMBINE suggestion with

RELATION	MEANING	DEFAULT SYSTEM ACTION
	provide a fuller description	proposed combined text
CONTRADICTION	Incompatible claims about the same aspect	HUMAN_REVIEW; no factual winner selected
INDEPENDENT	Different findings or semantic targets	KEEP_BOTH; does not require review by finalization gate

3.6 API ARCHITECTURE

Table 3.5 Major Inspection API Operations

METHOD	ENDPOINT GROUP	FUNCTION
POST	/inspections	Create an inspection
GET	/inspections	List accessible inspections
POST	/inspections/:id/share	Add a collaborator
POST	/inspections/:id/locations	Create a location
POST	/inspections/:id/observations/snapshot	Upsert observation snapshots
POST	/inspections/:id/semantic-analysis	Analyze observation pairs for a location
POST	/inspections/:id/semantic-results/:resultId/decide	Record human resolution
GET	/inspections/:id/report-status	Read review/finalization/export status
POST	/inspections/:id/finalize	Record collaborator finalization
GET	/inspections/:id/export-data	Return finalized observations and resolved semantic results

CHAPTER 4

PROJECT DESCRIPTION

4.1 METHODOLOGY

The project follows an incremental implementation methodology. The collaboration foundation is implemented first: authentication, document creation, TipTap editing, Yjs synchronization, offline persistence and server persistence. Diagnostics are then used to verify connectivity state, merge activity and latency. AI-assisted change summaries are layered on top without being allowed to change the replicated data model.

The semantic research component adopts a separation-of-concerns methodology. The approved paper conceptualizes a cost-sensitive cascade in which structural/lexical features, Sentence-BERT similarity and bidirectional NLI handle clear cases before selective LLM escalation and human verification [13]-[20]. The deployed Inspection feature currently implements the four-class policy through pair filtering and a schema-validated Gemini request. This is treated as the implementation state, while the fuller cascade remains a research-backed extension path rather than being falsely reported as production code.

The Inspection workflow was then integrated as a domain-specific application. Rather than storing only a free-form document, it creates explicit locations and authored observations. This lets the system restrict semantic comparison to the same physical location, display collaborator-specific contributions, and record a decision for each non-independent pair. Multi-user finalization acts as the final quality gate before export.

4.2 SYSTEM MODULES

4.2.1 Authentication Module

Supports email/password authentication, JWT-based sessions and Google OAuth. Protected REST operations use authenticated user identity to determine ownership and collaborator permissions.

4.2.2 Dashboard and Document Management Module

Presents recent documents, shared documents, inspection access and creation actions. Document APIs support ownership, sharing and room-level isolation.

4.2.3 Rich-Text Editing Module

Uses TipTap/ProseMirror to provide formatted collaborative text editing. Editor state is bound to a shared Yjs document rather than a server-owned text blob.

4.2.4 Collaboration and Awareness Module

Exchanges Yjs updates and user-awareness information over WebSocket. Collaborator presence and cursors remain separate from persisted document content.

4.2.5 Offline Persistence and Reconnection Module

Stores Yjs state in IndexedDB. Disconnection does not block local editing. Reconnection restores transport and synchronizes only missing CRDT state.

4.2.6 Backend and Database Module

Provides Express REST APIs, WebSocket handling and Mongoose persistence for users, documents, inspections, observations, semantic results and logs.

4.2.7 AI-Assisted Change Summary Module

Derives a human-readable explanation of changes since a previous synchronization point using the Gemini API. The summary is informational and does not replace the CRDT change history.

4.2.8 Inspection Workspace Module

Provides inspection creation, location tree, observation cards, collaborator panel, sharing, semantic reconciliation, status tracking, finalization and PDF export.

4.2.9 Semantic Reconciliation Module

Snapshots observations, generates candidate pairs, calls the semantic service, stores four-class results, exposes reasons and suggestions and records human decisions.

4.2.10 Report Finalization and Export Module

Blocks finalization while required reviews are pending, records each user approval, unlocks export only after all participants finalize and returns structured export data for PDF generation.

4.2.11 Diagnostics and Research Logging Module

Captures synchronization/consistency information for evaluation and records Inspection events such as creation, semantic analysis, human decisions and finalization for traceability.

4.3 DETAILED INSPECTION FEATURE

4.3.1 Inspection Workspace and Location Model

The Inspection workspace is displayed independently from the general editor so that structured field observations are not confused with ordinary document paragraphs. A left-side location tree allows the user to create and select locations such as floors, rooms or building zones. The central panel

shows observations associated with the selected location, while the right-side panel displays collaborators and report status.

Each stored location contains a `locationId`, display name, optional parent identifier, creator and creation time. The optional parent field allows future hierarchical location trees without requiring the database schema to change. A backup of the location list is retained on the Inspection document for server-side access even when the shared client state is unavailable.

4.3.2 Observation Capture and Connectivity Metadata

Every observation includes a stable `observationId`, `locationId`, author identity, author display name, plain-text content and connectivity state. Snapshot synchronization uses a bulk upsert so repeated snapshots update existing records rather than creating duplicates. Plain text is limited to 10,000 characters. If the client deletes an observation, the server removes both the stale observation snapshot and semantic results that refer to it, preventing an invalid review from remaining visible.

4.3.3 Complementary Observation Handling

When two observations describe the same local finding but contribute compatible information, the relationship is `COMPLEMENTARY`. The interface displays the confidence value, explanation and a suggested combined statement. The user can accept the recommendation without losing authorship context. In the captured A Block example, one observation identifies a crack on the left wall of Room 101 and the second describes the crack as light; the system therefore produces a combined sentence that retains both location and severity.

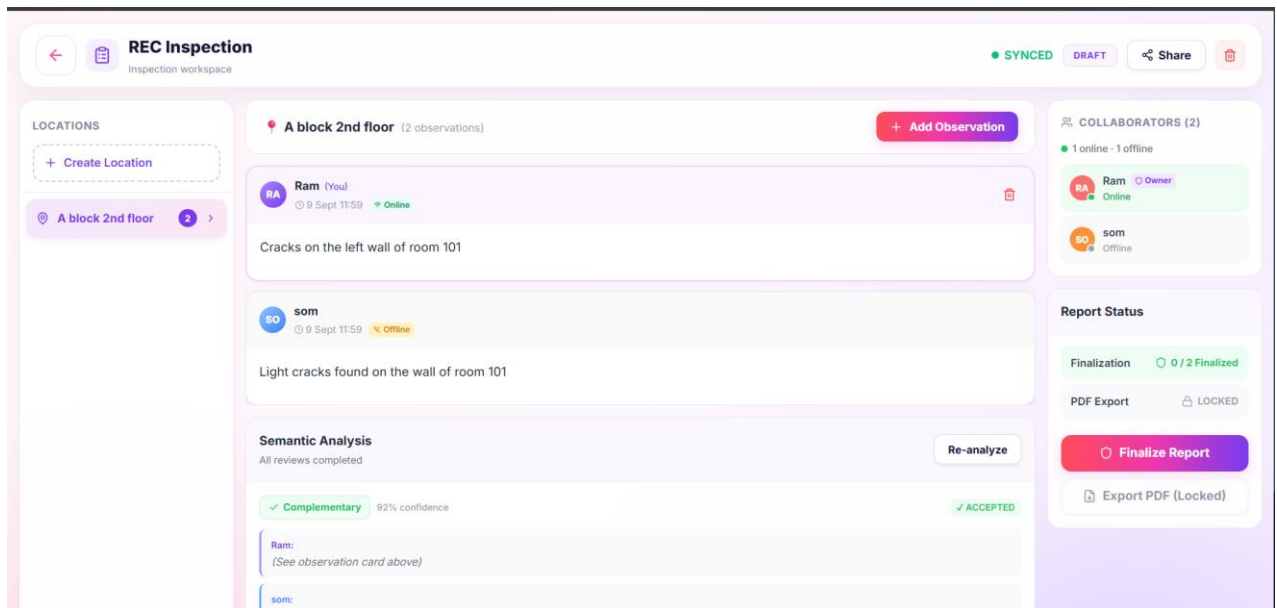


Figure 4.1 Inspection Workspace with Location-Based Observations

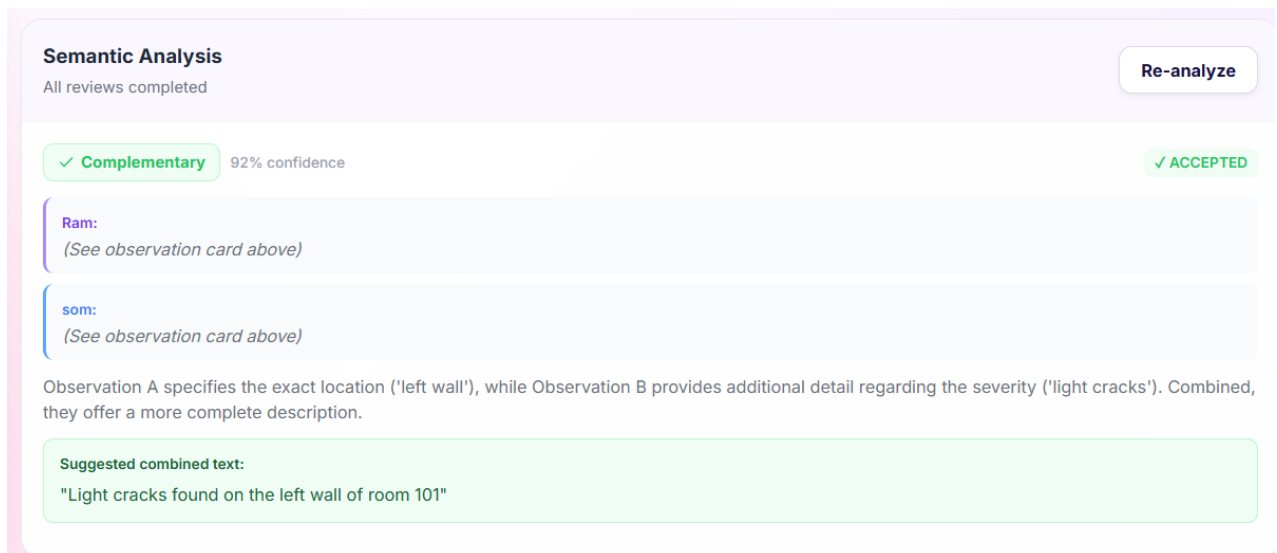


Figure 4.2 Complementary Semantic Analysis and Suggested Combined Text

4.3.4 Contradiction Handling and Human Decision

For a contradictory pair the system does not choose a factual winner. The semantic result explains the incompatible claims and marks the pair as a potential contradiction. A reviewer can retain both observations for later investigation or dismiss/resolve the relationship according to the available workflow. The human decision is stored together with the reviewer ID and decision timestamp, creating an auditable separation between machine classification and human responsibility.

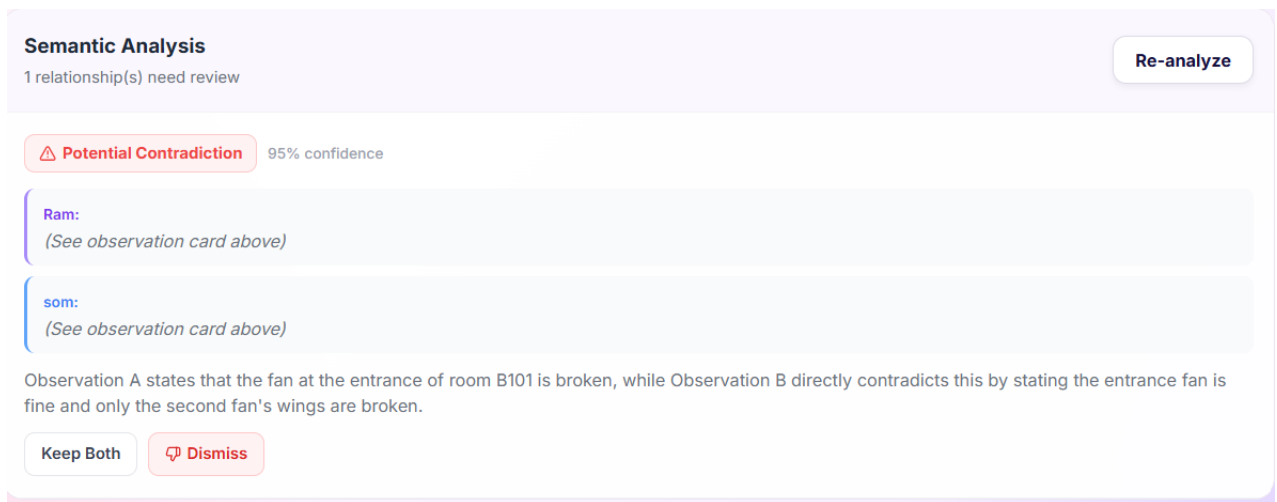


Figure 4.3 Potential Contradiction Requiring Human Review

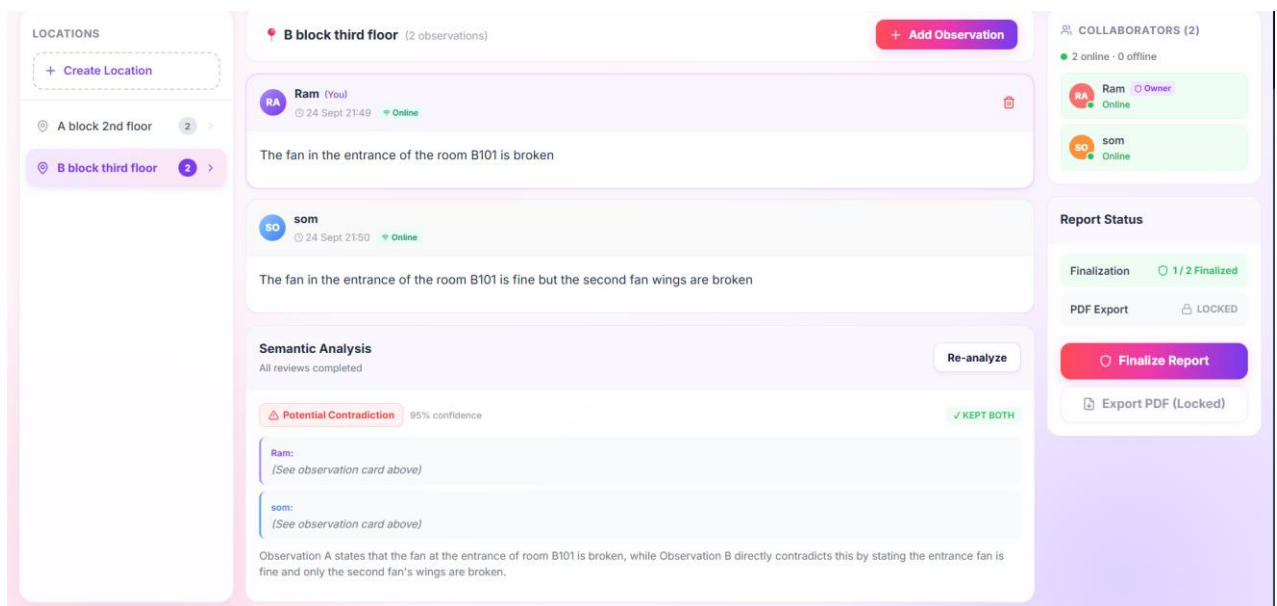


Figure 4.4 Contradictory Observations Retained by Human Decision

4.3.5 Collaborator Finalization and Export Lock

After semantic reviews are complete, each participant finalizes the inspection independently. The status panel presents a fraction such as “1 / 2 Finalized.” During partial finalization, PDF export remains locked. The backend uses the owner plus unique collaborators as the required user set, and duplicate collaborator identities are removed before the count is calculated.

When the number of finalized users reaches the required count, the inspection becomes FINALIZED and finalizedAt is stored. The report-status endpoint then returns pdfExport = AVAILABLE. The UI replaces the

finalization action with a confirmation that all collaborators have finalized and enables the Export PDF button. This gating prevents one collaborator from publishing the report while another still considers it unfinished.

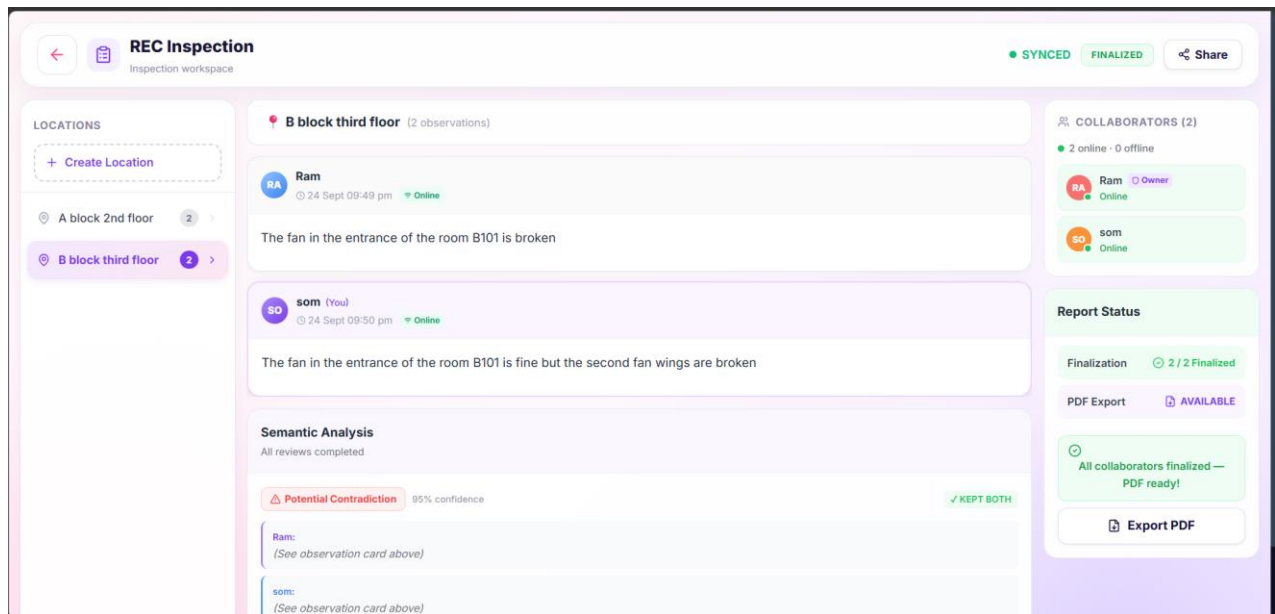


Figure 4.5 Complete Multi-User Finalization with PDF Export Enabled

4.3.6 Exported Inspection Report

The export-data endpoint returns the inspection record, observations sorted by location and creation time, and only semantic results whose humanDecision is not PENDING. Client-side PDF generation can therefore group content by location and include resolved semantic outcomes. The supplied sample export shows the finalized timestamp, a location with no finalized observations, and a B Block location containing two conflicting observations with the resolution “Both observations retained for review.”

Figure 4.6 Sample Exported Inspection PDF

REC Inspection

Finalized: 24/09/2026, 22:41:35

A block 2nd floor

No finalized observations.

B block third floor

Conflicting Observations

Ram

The fan in the entrance of the room B101 is broken

som

The fan in the entrance of the room B101 is fine but the second fan wings are broken

Resolution: Both observations retained for review.

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION

The integrated build contains authentication, dashboard navigation, rich-text collaboration, offline persistence, reconnection, sharing, AI change summaries, diagnostics, Inspection management, semantic reconciliation, multi-user finalization and PDF export. The implementation uses a common React/TypeScript frontend and Node.js/Express backend, while synchronization-sensitive document state is handled by Yjs rather than by ordinary REST writes.

The WebSocket layer is responsible for real-time document exchange and room-level broadcast events. Inspection semantic updates and status updates are broadcast to the corresponding inspection room so collaborators can refresh review state without waiting for manual reload. REST remains appropriate for durable application records such as observations, semantic decisions and finalization.

system.

Figure 5.1 Dashboard Interface

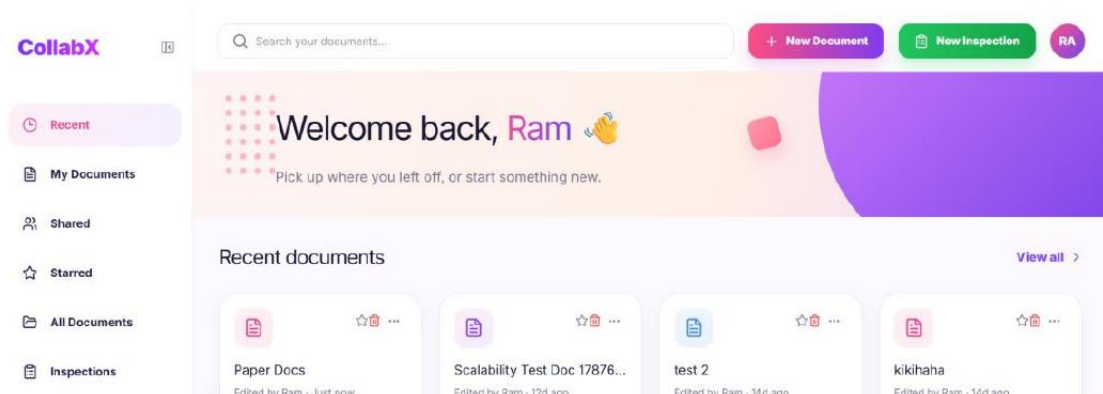


Figure 5.1 Dashboard Interface

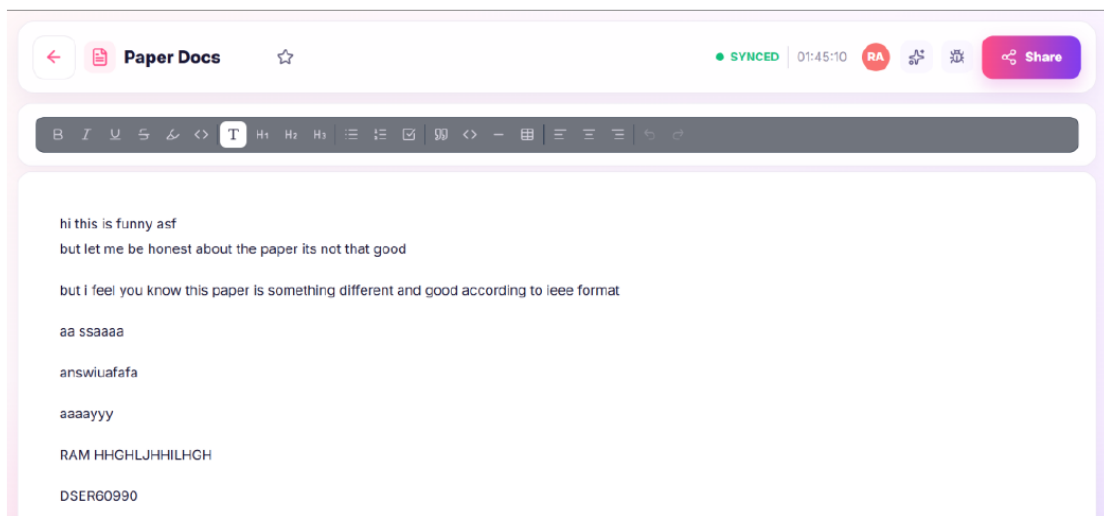


Figure 5.2 Collaborative Rich-Text Editor Interface

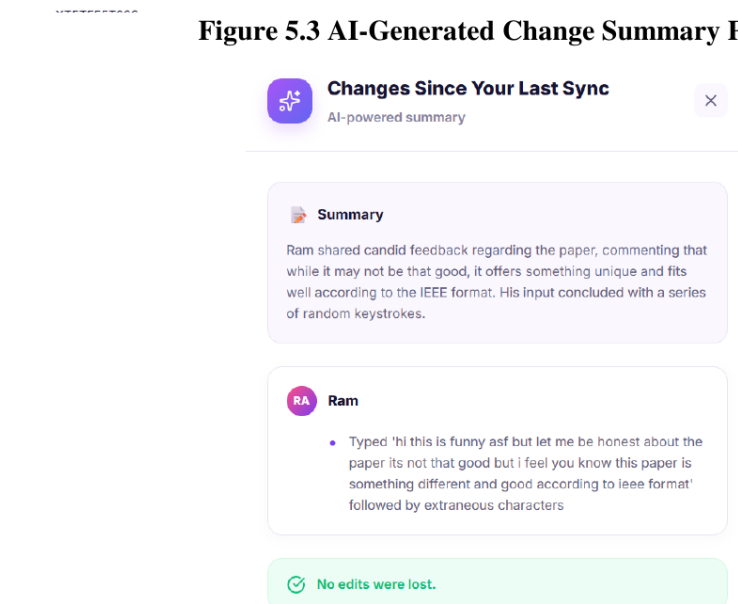


Figure 5.3 AI-Generated Change Summary Panel

Figure 5.4 Diagnostics Panel

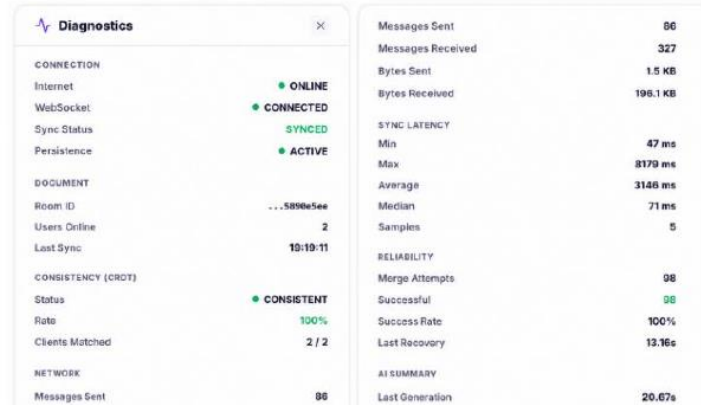


Figure 5.4 Diagnostics Panel

5.2 EXPERIMENTAL SETUP

The synchronization evaluation described in the existing draft used a local development server, desktop browser clients and a mobile browser on the local network. Wi-Fi and a mobile hotspot were used to exercise normal connectivity and interruption/recovery behavior. The recorded draft configuration used Yjs 13.6.31 on the client and 13.6.22 on the server, Google Chrome 151.0.7922.174, and a test laptop with a 13th Gen Intel Core i7-13650HX processor and 24 GB RAM. These measurements characterize the development setup and should not be interpreted as production capacity limits.

Table 5.1 Test Environment Configuration

PARAMETER	VALUE
Browsers	Google Chrome and Microsoft Edge; mobile browser over local network
Network	Wi-Fi, mobile hotspot and simulated disconnect/reconnect
Yjs versions	Client 13.6.31; server 13.6.22
Test browser	Google Chrome 151.0.7922.174
Test machine	13th Gen Intel Core i7-13650HX; 24 GB RAM
Monitoring	In-editor diagnostics: connectivity, consistency, latency and message counts

5.3 COLLABORATION TEST SCENARIOS

Table 5.2 Collaboration and Offline Test Scenarios

ID	SCENARIO	PURPOSE
T1	Simultaneous editing	Verify real-time collaborative updates
T2	Multiple concurrent edits	Verify preservation of independent changes
T3	Single-client offline editing	Verify uninterrupted local editing
T4	Multiple clients offline	Verify independent offline changes
T5	Reconnection and synchronization	Verify recovery and state convergence
T6	Browser refresh while offline	Verify IndexedDB persistence
T7	Close and reopen while offline	Verify persisted restoration
T8	Network interruption during synchronization	Verify repeated recovery
T9	Concurrent editing in the same area	Observe CRDT conflict handling
T10	Increasing concurrent users	Observe behavior at 2/5/10/20 simulated users
T11	Different document rooms	Verify room isolation

5.4 SYNCHRONIZATION RESULTS

Under normal connectivity, edits were reflected on collaborating clients without a page refresh. The diagnostics panel reported the document state as consistent and the draft records a median synchronization latency of 71 ms during the test session. Across the evaluated offline/reconnection scenarios, 98 merge attempts were recorded and all completed successfully, corresponding to a 100% merge success rate in the reported test set.

Offline persistence was verified by continuing to edit a disconnected client, refreshing the browser while offline and closing/reopening the document before reconnection. Locally entered content remained available. When connectivity returned, offline and remote changes were merged and the diagnostics status returned to synchronized/consistent. A separate room-isolation test confirmed that updates from one document room did not appear in another.

The scalability script tested 2, 5, 10 and 20 simulated users. The measured averages decreased from 44.51 ms at two users to 12.68 ms at twenty users. The draft correctly treats this as an observed characteristic of the local test configuration rather than evidence that adding users improves latency.

Table 5.3 Scalability Test Results from the Existing Draft

CONCURRENT USERS	AVG. SYNC LATENCY	MESSAGES SENT	MESSAGES RECEIVED	DATA SENT	DATA RECEIVED
2	44.51 ms	48	53	1.21 KB	3.74 KB
5	29.74 ms	120	505	3.02 KB	24.53 KB
10	18.52 ms	240	2,225	6.04 KB	96.96 KB
20	12.68 ms	480	9,179	12.29 KB	465.83 KB

5.5 INSPECTION FEATURE FUNCTIONAL VALIDATION

Table 5.4 Inspection Workflow Functional Validation

ID	TEST	EXPECTED BEHAVIOR	EVIDENCE / RESULT
I1	Create inspection and location	Location appears in the location tree	Supported by implemented CRUD and captured UI
I2	Two collaborators add observations	Both authored cards shown with online/offline state	Observed in supplied screenshots
I3	Complementary observations	Relationship, confidence, explanation and combined text displayed	Observed: 92% complementary example
I4	Contradictory observations	Potential contradiction displayed without choosing factual winner	Observed: 95% contradiction example
I5	Human resolution	Reviewer decision persisted and status reflected	Observed “KEPT BOTH” state; controller persists decision metadata
I6	Partial finalization	Finalization count updates but PDF stays locked	Observed 1/2 Finalized with export locked
I7	All users finalized	Inspection becomes finalized and export becomes available	Observed 2/2 Finalized with Export PDF enabled
I8	PDF generation	Export groups observations by location and includes resolution	Confirmed by supplied sample export PDF

The captured A Block example demonstrates the complementary path. Two collaborators contribute related descriptions of a crack in Room 101. The

semantic panel classifies the pair as complementary, explains that one statement contributes exact location and the other contributes severity, and proposes a combined sentence. This matches the implemented COMBINE action and shows how AI assistance can improve completeness without discarding either observation.

The B Block example demonstrates the safety-critical contradiction path. One collaborator states that the entrance fan is broken, while another states that the entrance fan is fine and a different fan is damaged. The interface reports a potential contradiction with 95% confidence and explains the incompatibility. The user retains both observations, and that human outcome is later represented in the exported PDF. The system therefore exposes disagreement instead of silently rewriting it.

The finalization screenshots validate the approval gate. At 1/2 finalized, report status shows PDF Export as locked. At 2/2 finalized, the workspace is marked FINALIZED, the status message indicates that all collaborators have finalized and the Export PDF control becomes available. This behavior corresponds directly to the backend report-status and finalizeReport logic.

5.6 SEMANTIC RESEARCH EVALUATION FROM THE APPROVED PAPER

The approved conference paper evaluates the broader semantic-triage framework on a separate 300-pair, two-annotator benchmark. That evaluation is not identical to the deployed Gemini-only Inspection service and is therefore reported separately rather than being presented as a production metric. The paper’s full-cascade evaluation reported Cohen’s kappa of 0.800, 80.7% LLM-avoidance coverage, 87.3% contradiction recall and 75.7% four-class classification accuracy.

The contradiction-recall result is particularly relevant to the project’s policy because missed contradictions are more consequential than unnecessary

review prompts. The research design therefore favors conservative escalation. The implemented Inspection feature follows the same governance principle even though its current inference path is simpler: contradictions remain human-review items and the model is prohibited from selecting the “true” inspector statement.

Table 5.5 Approved-Paper Semantic Triage Evaluation

METRIC	REPORTED VALUE	INTERPRETATION
Two-annotator agreement (Cohen’s kappa)	0.800	Substantial agreement in the 300-pair evaluation
Classification accuracy	75.7%	Overall four-class performance of the evaluated cascade
Contradiction recall	87.3%	Sensitivity to the safety-critical contradiction class
LLM-avoidance coverage	80.7%	Share of pairs handled without LLM use in the paper’s cascade

5.7 DISCUSSION

The results support the architectural separation used by the project. Yjs solves the availability and convergence problem, while the semantic layer addresses a different class of content risk after observations have been collected. Because the semantic service cannot modify Yjs merge semantics, a temporary AI failure does not prevent users from synchronizing their work. The controller also continues with remaining observation pairs if one AI call fails.

The Inspection workflow adds governance that a general editor does not normally provide. Semantic decisions are durable records, finalization is user-specific, and export is a collective action. This is important in reports where collaborators may be professionally accountable for their observations. The sample PDF shows that unresolved factual disagreement can remain visible as a retained conflict rather than being obscured by generated prose.

The principal limitation is that the final implementation does not yet deploy every component of the research paper’s lightweight cascade. The production semantic service currently relies on pair filtering plus Gemini, so its cost, calibration and generalization must be assessed separately from the approved paper’s SBERT/NLI/LLM evaluation. The current report therefore keeps research results and implementation validation distinct.

5.8 LIMITATIONS

- The synchronization measurements come from a development environment and do not establish production-scale capacity.
- The highest reported load-test concurrency was 20 simulated users; longer-running sessions and larger documents were not exhaustively evaluated.
- Network evaluation focused on disconnection/reconnection and did not systematically characterize sustained packet loss, extreme latency or prolonged low-bandwidth conditions.
- The implemented Inspection semantic service depends on an external Gemini API and may vary across model versions, prompts and service availability.
- Confidence returned by the model is an application aid rather than a statistically calibrated probability unless separately calibrated on domain data.
- Pairwise comparison is location-local and may miss long-range semantic dependencies between observations in different locations or across a long report.
- The supplied functional screenshots demonstrate representative Inspection behavior; a larger user study is still required to evaluate usability and review workload in real field inspections.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project designed and implemented an offline-first collaborative platform in which document availability and replica convergence are handled by Yjs CRDTs, while application-level semantics remain a separate concern. Local IndexedDB persistence enables continued editing during disconnection, WebSocket synchronization restores shared state after reconnection, and MongoDB provides server-side persistence and queryable application data.

The major final-stage addition is the Inspection workspace. It converts the semantic-conflict concept from the approved paper into a concrete collaborative workflow built around locations, authored observations, four-class semantic analysis, human decisions, participant finalization and controlled PDF export. The system does not allow the AI to declare which inspector is factually correct. Instead, it classifies the relationship, explains its reasoning, suggests a minimally invasive action and preserves human authority over contradictions.

The implemented finalization rule further strengthens accountability: semantic review must be complete before users can finalize, and the report cannot be exported until all required participants have finalized. The sample export confirms that the resulting PDF can preserve both observations and the selected resolution. Within the evaluated development environment, the existing project tests also reported reliable offline recovery, 100% recorded merge success and consistent final replicas. The project therefore demonstrates how CRDT synchronization, AI assistance and human review can coexist without conflating state convergence with semantic correctness.

6.2 FUTURE WORK

- Integrate the full lightweight cascade described in the approved paper, using structural filtering, Sentence-BERT similarity and bidirectional NLI before selective LLM escalation.
- Calibrate confidence and routing thresholds using a larger inspection-specific benchmark rather than relying only on general-language examples.
- Add richer evidence handling for factual disputes, including photographs, annotations, measurements, timestamps and links to inspection standards.
- Extend semantic analysis from pairwise location-local observations to longer dependency chains and cross-location consistency checks.
- Provide configurable roles for inspector, reviewer and approving authority, with digital signatures and immutable audit export where required.
- Perform field studies with real inspection teams to measure review workload, user trust, false-positive burden and time-to-final-report.
- Expand scalability and network testing to larger user counts, larger documents, longer offline intervals, packet loss and variable bandwidth.
- Explore on-device or privacy-preserving semantic models for deployments where inspection text cannot be sent to a third-party model service.
- Investigate hybrid or peer-to-peer synchronization to reduce dependence on a central WebSocket relay while preserving persistence and access control.

APPENDIX 1

INSPECTION DATA MODEL AND STATUS RULES

This appendix summarizes implementation details extracted from the final project source code. It is included to document the exact final-stage Inspection behavior without overloading the main chapters.

Table A.1 Inspection Rules Implemented in Source Code

AREA	IMPLEMENTED RULE
Inspection status	DRAFT, SYNCHRONIZING, REVIEW_REQUIRED, READY_TO_FINALIZE, FINALIZED
Observation connectivity	online or offline stored with observation snapshot
Semantic relationship	DUPLICATE, COMPLEMENTARY, CONTRADICTORY, INDEPENDENT
Human decision	PENDING, ACCEPTED, REJECTED, KEPT_BOTH, KEPT_ONE, EDITED
Candidate filtering	Observation text must contain at least five trimmed characters
AI retry	Up to five retries for HTTP 503 with exponential backoff
Finalization prerequisite	No PENDING non-independent semantic result
Complete finalization	Owner plus all unique collaborators recorded in finalizedBy
PDF availability	Only when inspection is effectively finalized
Export contents	Inspection metadata, observations and semantic results with non-pending decisions

APPENDIX 2

DETAILED TEST CHECKLIST

Table B.1 End-to-End Project Verification Checklist

ID	AREA	CHECK
B1	Authentication	Register/login and open protected dashboard
B2	Document creation	Create a new collaborative document
B3	Real-time edit	Open same document in two clients and verify remote update
B4	Offline edit	Disconnect one client and continue typing
B5	IndexedDB persistence	Refresh/close while offline and verify local content remains
B6	Reconnection	Restore network and verify CRDT convergence
B7	Room isolation	Edit two independent document rooms and verify no cross-room updates
B8	Inspection creation	Create an inspection and verify owner access
B9	Share inspection	Invite collaborator with editor/viewer permission
B10	Location creation	Create multiple inspection locations
B11	Observation snapshot	Add observations and verify stored author/connectivity metadata
B12	Complementary classification	Create compatible observations and verify combined text suggestion
B13	Contradiction classification	Create incompatible observations and verify human-review prompt
B14	Human decision	Record accepted/kept-both/other resolution and verify persistence
B15	Review gate	Leave a required result pending and verify finalization is blocked
B16	Partial finalization	Finalize one user and verify PDF export remains locked
B17	Complete finalization	Finalize all users and verify FINALIZED status
B18	PDF export	Export and verify locations, observations and resolved conflicts appear
B19	Deletion cleanup	Delete observation and verify related stale semantic results are removed
B20	Research log	Verify analysis/decision/finalization events are logged

REFERENCES

- [1] C. Sun and C. A. Ellis, “Operational transformation in real-time group editors: issues, algorithms, and achievements,” in Proc. ACM Conf. Computer Supported Cooperative Work (CSCW), 1998, pp. 59-68. doi: 10.1145/289444.289469.
- [2] N. Pregoica, J. M. Marques, M. Shapiro, and M. Letia, “A commutative replicated data type for cooperative editing,” in Proc. 29th IEEE Int. Conf. Distributed Computing Systems (ICDCS), 2009, pp. 395-403. doi: 10.1109/ICDCS.2009.20.
- [3] M. Shapiro, N. Pregoica, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in Proc. 13th Int. Conf. Stabilization, Safety, and Security of Distributed Systems (SSS), 2011, pp. 386-400. doi: 10.1007/978-3-642-24550-3_29.
- [4] M. Kleppmann and A. R. Beresford, “A conflict-free replicated JSON datatype,” IEEE Trans. Parallel Distrib. Syst., vol. 28, no. 10, pp. 2733-2746, 2017. doi: 10.1109/TPDS.2017.2697382.
- [5] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming Software (Onward!), 2019, pp. 154-178. doi: 10.1145/3359591.3359737.
- [6] N. Schiefer, G. Litt, and D. Jackson, “Merge what you can, fork what you can’t: managing data integrity in local-first software,” in Proc. 9th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC), 2022, pp. 24-32. doi: 10.1145/3517209.3524041.
- [7] G. Litt, S. Lim, M. Kleppmann, and P. van Hardenberg, “Peritext: A CRDT for collaborative rich text editing,” Proc. ACM Hum.-Comput. Interact., vol. 6, no. CSCW2, Art. 531, 2022. doi: 10.1145/3555644.
- [8] J. Gentle and M. Kleppmann, “Collaborative text editing with Eg-walker: Better, faster, smaller,” in Proc. 20th European Conf. Computer Systems (EuroSys), 2025, pp. 311-328. doi: 10.1145/3689031.3696076.
- [9] M. Weidner, J. Gentle, and M. Kleppmann, “The art of the Fugue: Minimizing interleaving in collaborative text editing,” IEEE Trans. Parallel Distrib. Syst., vol. 36, no. 11, pp. 2425-2437, 2025. doi: 10.1109/TPDS.2025.3611880.
- [10] G. Semenov and V. Aksenov, “Semantic conflict model for collaborative data structures,” in Proc. 13th Int. Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC), 2026, pp. 25-31. doi: 10.1145/3806077.3806700.
- [11] N. Gu, J. Xu, X. Wu, J. Yang, and W. Ye, “Ontology based semantic conflicts resolution in collaborative editing of design documents,” Advanced Engineering Informatics, vol. 19, no. 2, pp. 103-111, 2005. doi: 10.1016/j.aei.2005.05.005.
- [12] D. Yang, A. Halfaker, R. Kraut, and E. Hovy, “Identifying semantic edit intentions from revisions in Wikipedia,” in Proc. 2017 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 2000-2010. doi: 10.18653/v1/D17-1213.
- [13] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, “A large annotated corpus for learning natural language inference,” in Proc. 2015 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 632-642. doi: 10.18653/v1/D15-1075.

- [14] A. Williams, N. Nangia, and S. R. Bowman, “A broad-coverage challenge corpus for sentence understanding through inference,” in Proc. NAACL-HLT, 2018, pp. 1112-1122. doi: 10.18653/v1/N18-1101.
- [15] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992. doi: 10.18653/v1/D19-1410.
- [16] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, “FEVER: a large-scale dataset for fact extraction and verification,” in Proc. NAACL-HLT, 2018, pp. 809-819. doi: 10.18653/v1/N18-1074.
- [17] M. Lee, P. Liang, and Q. Yang, “CoAuthor: Designing a human-AI collaborative writing dataset for exploring language model capabilities,” in Proc. CHI Conf. Human Factors in Computing Systems, 2022, Art. 388, pp. 1-19. doi: 10.1145/3491102.3502030.
- [18] Q. Ruan, I. Kuznetsov, and I. Gurevych, “Re3: A holistic framework and dataset for modeling collaborative document revision,” in Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024, pp. 4635-4655. doi: 10.18653/v1/2024.acl-long.255.
- [19] L. Gao et al., “RARR: Researching and revising what language models say, using language models,” in Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL), 2023, pp. 16477-16508. doi: 10.18653/v1/2023.acl-long.910.
- [20] P. Manakul, A. Liusie, and M. J. F. Gales, “SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models,” in Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 9004-9017. doi: 10.18653/v1/2023.emnlp-main.557.